

使用 Gemini Code Assist 探索及強化 AI 摘要快速部署解決方案

來源：<https://codelabs.developers.google.com/codelabs/gemini-code-assist-developer-aisummarization-jss?hl=zh-tw>

步驟數：13

在這個程式碼研究室中，我們會說明現有的快速部署解決方案「AI 摘要」，這個解決方案使用 Vertex AI 模型為已上傳至 Google Cloud Storage 的 PDF 文件產生摘要。我們將使用 Gemini Code Assist，瞭解並新增功能到解決方案。

目錄

1. 簡介
2. 設定
3. 部署 AI 摘要快速部署解決方案
4. 與 Gemini 對話
5. 在 Cloud Code 中下載 Jump Start 解決方案 Cloud 函式
6. 查看現有專案
7. 執行範例執行作業
8. 為應用程式建立網頁應用程式用戶端
9. 在本機執行網頁應用程式
10. (選用) 開放式探索 - 部署至 Cloud Run
11. (選用) 開啟「探索 - 新增 CSS 樣式」
12. 恭喜！
13. 參考文件

1. 簡介

在本程式碼研究室中，我們將瞭解現有的快速入門解決方案「AI 摘要」，該解決方案使用 Vertex AI 模型，為上傳至 Google Cloud Storage 的 PDF 文件產生摘要。

接著，我們將使用 Gemini Code Assist 執行下列操作：

- 瞭解 Python 程式碼，這些程式碼會驅動 Cloud Function 從 PDF 文件中擷取文字、摘要內容，並將結果寫入 BigQuery。
- 我們會在整個過程中運用 Gemini Code Assist，協助編寫新功能。我們會開發網頁應用程式 (Python Flask 應用程式)，並在本機執行應用程式來驗證程式碼。
- 我們也可以選擇將這個應用程式部署至 Cloud Run，並使用 Material Design 改善網頁應用程式的設計 (美觀)。

執行步驟

- 您將部署 AI 摘要快速部署解決方案，並觸發程序流程，瞭解其運作方式。
- 接著，您將使用 Cloud Shell IDE 下載 Jump Start 解決方案的現有程式碼，並使用 Gemini Code Assist 瞭解程式碼。
- 您將使用 Gemini Code Assist Cloud Shell IDE，為新功能生成程式碼。

課程內容...

- 瞭解 AI 摘要快速部署解決方案的運作方式。
- 如何使用 Gemini Code Assist 執行多項開發人員工作，例如生成程式碼、補全程式碼及產生程式碼摘要。

事前準備

- Chrome 網路瀏覽器
- Gmail 帳戶
- 已啟用計費功能的 Cloud 專案
- 為雲端專案啟用 Gemini Code Assist

本實驗室適合各種程度的開發人員，包括初學者。雖然範例應用程式是以 Python 語言編寫，但您不需要熟悉 Python 程式設計，也能瞭解發生了什麼事。我們將著重於熟悉 Gemini Code Assist 的開發人員功能。

步驟 2 · 約 5 分鐘

2. 設定

本節說明如何開始進行這個實驗室。

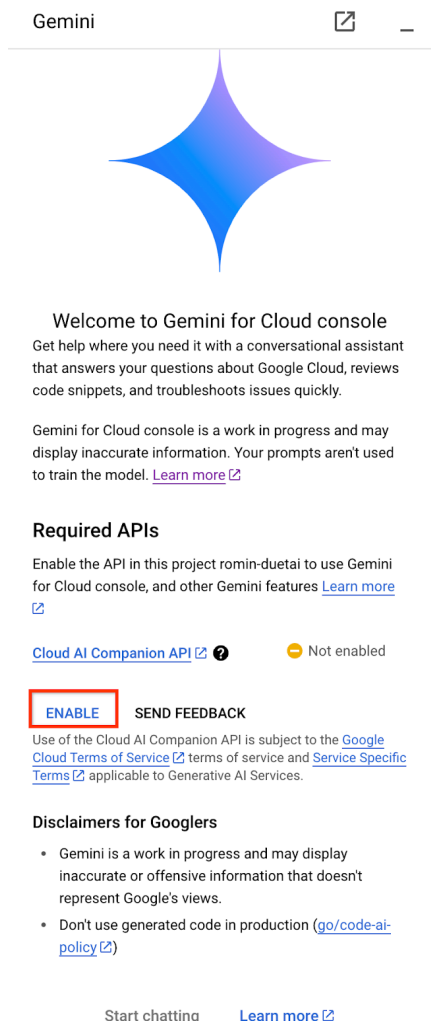
在 Google Cloud 專案中啟用 Gemini for Cloud

我們現在要在 Google Cloud 專案中啟用 Gemini for Cloud。請按照下列步驟操作：

1. 前往 <https://console.cloud.google.com>，並確認已選取要在本實驗室中使用的 Google Cloud 專案。按一下右上方的「開啟 Gemini」圖示。



2. 控制台右側會開啟 Gemini for Cloud 對話視窗。點選「啟用」按鈕，如下所示。如果沒有看到「啟用」按鈕，而是看到 Chat 介面，表示您可能已為專案啟用 Gemini for Cloud，可以直接前往下一個步驟。



3. 啟用後，您可以詢問一兩個問題，測試 Gemini for Cloud。畫面會顯示幾個查詢範例，但您可以嘗試類似 **What is Cloud Run?** 的查詢

Gemini is an AI-powered collaborator to help you get more done faster. Get answers to your questions about how to get started with a Cloud solution, strategies for optimizing resources, or using the gcloud CLI to manage Google Cloud.

In addition to general knowledge about Google Cloud, it also has some awareness of your context, like your project and console page.

What is Firestore?

Can I use Pub/Sub across different Google Cloud projects?

How does GKE scale my clusters?

Enter a prompt here



For best results use a detailed prompt. [Prompt guide](#)

Googler use of Gemini for Cloud console is subject to Google's [policy for using generative AI tools internally](#), and the [Google Employee Privacy Policy](#)

Gemini for Cloud 會回覆問題的答案。如要關閉 Gemini for Cloud 即時通訊視窗，請按一下右上角的

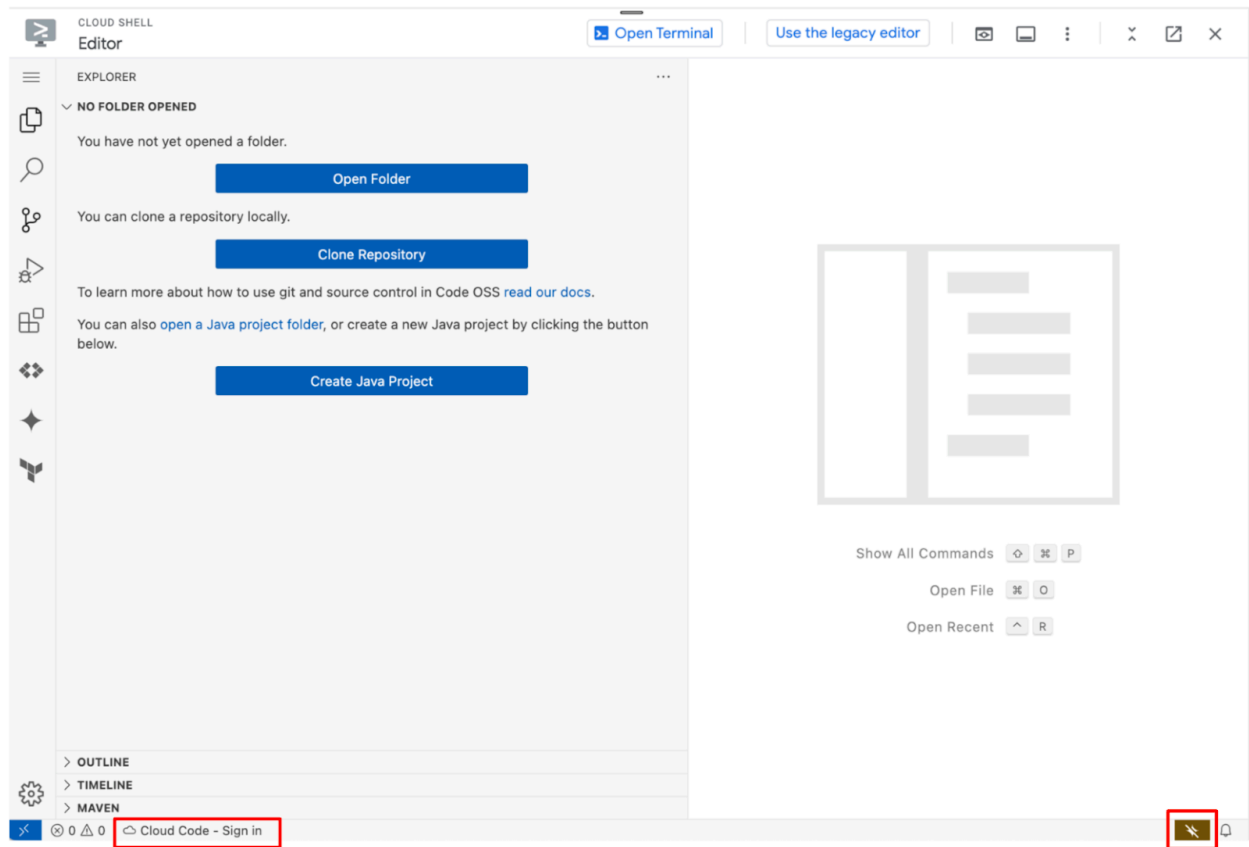
—

圖示。

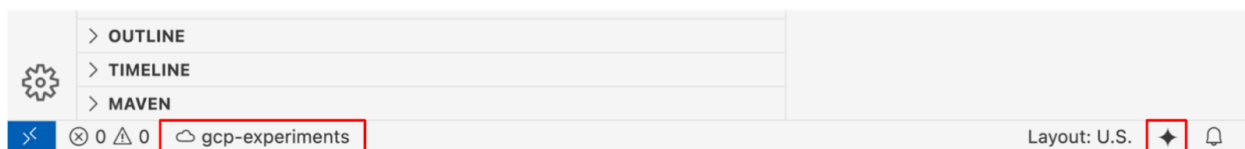
在 Cloud Shell IDE 中啟用 Gemini Code Assist

在接下來的程式碼研究室中，我們將使用 Cloud Shell IDE，這是以 [Code OSS](#) 為基礎的全代管開發環境。我們需要在 Cloud Shell IDE 中啟用及設定 Code Assist，步驟如下：

1. 前往 ide.cloud.google.com。IDE 可能需要一段時間才會顯示，請耐心等待。
2. 點選底部狀態列中的「Cloud Code - Sign in」按鈕，如下圖所示。依指示授權外掛程式。如果狀態列顯示「Cloud Code - no project」，請選取該項目，然後從專案清單中選取要使用的特定 Google Cloud 專案。



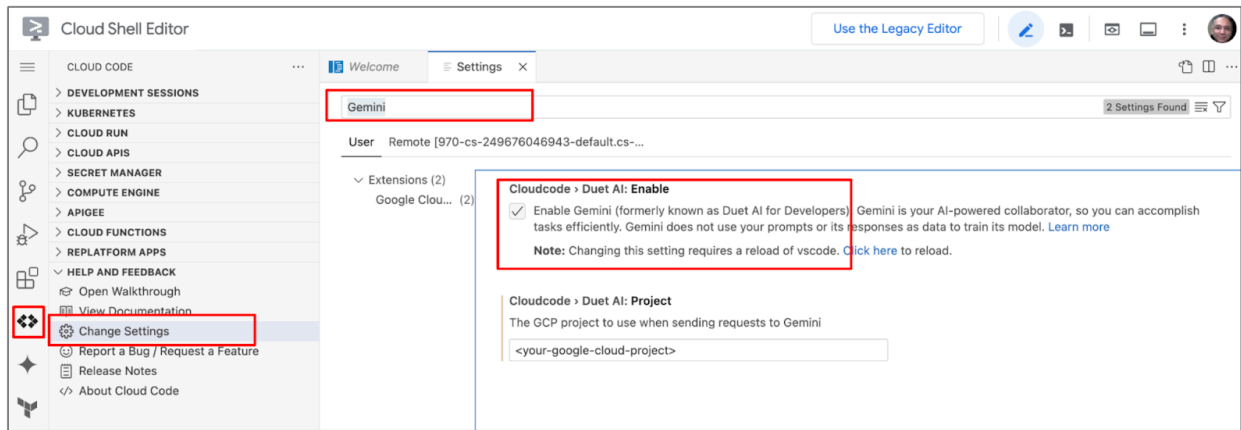
3. 如圖所示，按一下右下角的「Gemini」 **Gemini** 按鈕，然後最後一次選取正確的 Google Cloud 專案。如果系統要求啟用 **Cloud AI Companion API**，請啟用並繼續操作。
4. 選取 Google Cloud 專案後，請確認狀態列的 Cloud Code 狀態訊息中已顯示該專案，且狀態列右側也已啟用 Code Assist，如下所示：



現在可以開始使用 Gemini Code Assist 了！

您也可以使用 [Visual Studio Code](#) 或 [JetBrains IntelliJ](#) 等本機 IDE 中使用 Gemini Code Assist。Google Cloud Workstations 也提供這項功能。本程式碼研究室著重於使用 Cloud Shell IDE，讓所有人都使用相同的環境，但您也可以嘗試慣用的設定，詳情請參閱[這個連結](#)。

選用：如果右下方的狀態列未顯示 Gemini，請在 Cloud Code 中啟用 Gemini。請先前往「Cloud Code Extension」→「Settings」，然後輸入「Gemini」，確認 IDE 已啟用 Gemini，如下所示。確認已選取核取方塊。您應重新載入 IDE。這會啟用 Cloud Code 中的 Gemini，且狀態列中的 Gemini 圖示會顯示在 IDE 中。



步驟 3 · 約 15 分鐘

3. 部署 AI 摘要快速部署解決方案

1. 前往[生成式 AI 文件摘要解決方案](#)

2. 按一下「Deploy」(部署)

- 如果專案未啟用計費功能，請啟用計費功能。
 - 選取「us-central1」做為區域。
 - 按一下「部署」。
 - 這項作業最多可能需要 15 分鐘才能完成。
 - 您不需要進行任何變更，但歡迎點選解決方案部署詳細資料頁面中的「EXPLORE THIS SOLUTION」(探索這項解決方案) 按鈕，探索快速部署解決方案。
-

步驟 4 · 約 2 分鐘

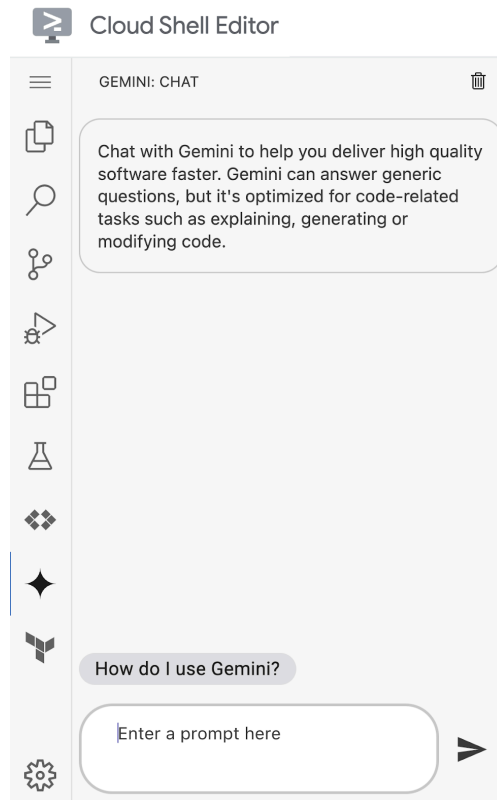
4. 與 Gemini 對話

首先，我們將學習如何與 Gemini 對話。在 VS Code 中，Gemini 可做為 Cloud Shell IDE 內的對話助理，是 Cloud Code 擴充功能的一部分。只要點選左側導覽列中的 Gemini 按鈕，即可開啟側邊面板。在左側導覽工具列中尋找 Gemini 圖示



，然後點選該圖示。

這會在 Cloud Shell IDE 中開啟「Chat: Gemini」窗格，您可以與 Gemini 對話，取得 Google Cloud 相關協助。



我們來使用 Gemini Chat 窗格輸入提示，並查看 Gemini 的回覆。輸入下列提示詞：

What is Cloud Run?

Gemini 應回覆 Cloud Run 的詳細資料。提示詞是指透過問題或陳述來說明您需要的協助。提示詞中可加入現有程式碼的背景資訊，Google Cloud 會分析這些資訊，提供更實用或完整的回覆。如要進一步瞭解如何撰寫提示詞才能生成好的回覆，請參閱「[為 Gemini in Google Cloud 撰寫更有效的提示詞](#)」。

請嘗試使用下列範例提示詞或任何您自己的提示詞，詢問有關 Google Cloud 的問題：

- What is the difference between Cloud Run and Cloud Functions?
- What services are available on Google Cloud to run containerized workloads?
- What are the best practices to optimize costs while working with Google Cloud Storage?

請注意頂端的垃圾桶圖示，這是重設 Code Assist 聊天記錄環境的方法。另請注意，這項即時通訊互動會根據您在 IDE 中處理的檔案提供相關資訊。

步驟 5 · 約 5 分鐘

5. 在 Cloud Code 中下載 Jump Start 解決方案 Cloud 函式

假設您位於 Cloud Shell 編輯器中，請按照下列步驟操作：

- 按一下 Cloud Code



- 注意：視螢幕大小而定，可能需要一個或兩個步驟。



或



- 按一下「Cloud Functions」。
- 如果出現提示，請登入或授權帳戶。
- 按一下 Webhook 函式。
- 按一下「下載至新工作區」圖示



-

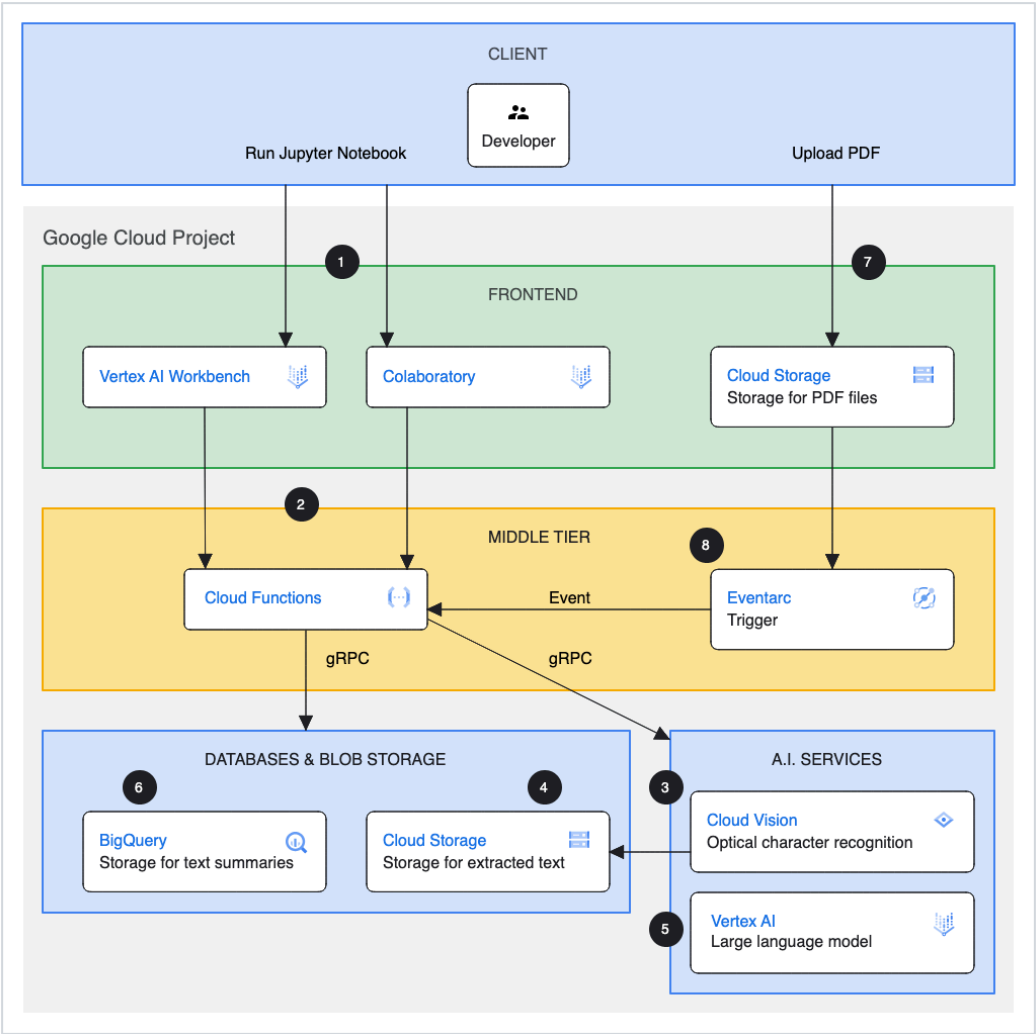
>  webhook us-central1



- 使用 webhook-1 做為工作區名稱 (應為預設名稱) 或其他名稱，然後按一下「確定」。
 - 這會在 Cloud Shell IDE 中開啟程式碼。
-

6. 查看現有專案

這項快速部署解決方案如下所示：



查看從「上傳 PDF」功能到 Cloud Storage 的流程。如果上傳 PDF 檔案，系統會叫用 `main.py` 檔案中指定的 Cloud Function。

按一下該檔案，雲端函式的進入點是 `entrypoint` 函式，最終會叫用 `cloud_event_entrypoint` 函式從 PDF 擷取文字，然後叫用 `summarization_entrypoint`，該函式會使用 Vertex AI 模型摘要內容，並將結果分別寫入 GCS 和 BigQuery。

反白選取 `main.py` 檔案中的所有程式碼，或任何特定程式碼片段。點選 Gemini Chat，然後輸入下列提示：`Explain this`。

系統隨即會提供程式碼說明。

7. 執行範例執行作業

根據架構圖，我們將檔案上傳至 **<PROJECT_ID>_uploads** bucket，以叫用 Cloud Function。

請確認您已準備好要上傳的 PDF 範例，並希望系統為該檔案產生摘要。

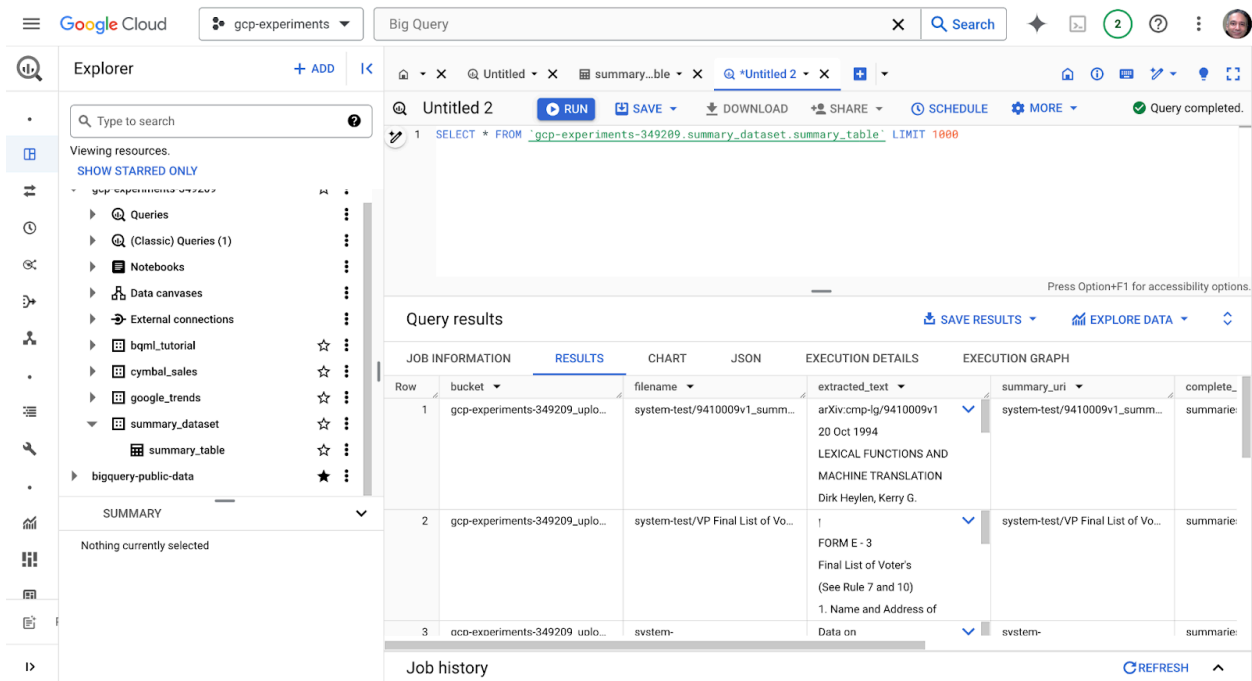
- 前往 Cloud Console 中的 [Google Cloud Storage](#)。
- 前往 **<PROJECT_ID>_uploads** 值區。按一下「上傳檔案」連結，然後上傳 PDF 範例。

如果沒有 PDF 範例檔案，可以使用我們建立的範例。在 Cloud Shell 中執行下列指令：

```
gsutil cp \
  gs://arxiv-dataset/arxiv/cmp-lg/pdf/9410/9410009v1.pdf \
  gs://<PROJECT_ID>_uploads/
```

檔案上傳成功後，系統會叫用 **webhook** 雲端函式，並摘要文件中找到的文字。輸出內容會寫入名為 **summary_dataset** 的 BigQuery 資料集，以及 **summary_table** 資料表。

查詢資料表，即可查看摘要結果。



8. 為應用程式建立網頁應用程式用戶端

上述程序是手動步驟，可逐步上傳要摘要的 PDF。要不要為應用程式建構網頁前端？

網頁前端的要求很簡單：

1. 基本 HTML 表單，可供我們選取及上傳需要摘要的檔案。
2. 成功上傳檔案後，檔案應會寫入 **<PROJECT_ID>_uploads** bucket，這樣其餘功能才能正常運作。

我們將使用 Python 和 Flask 網頁應用程式架構，在 Duet AI 的協助下建構這項功能。

讓我們開始吧！我們假設您仍在 Cloud Shell IDE 中開啟相同的工作區。

關閉所有檔案，然後在 Gemini Chat 視窗中輸入下列提示：

```
Write a Python Flask application that has a couple of routes:  
The root route / should serve the index.html page using the render_template framework. The  
/upload route should accept a file being uploaded and write that file to a Cloud Storage  
bucket.
```

理想情況下，這應該會產生如下程式碼：

```
from flask import Flask, render_template, request, redirect, url_for  
from google.cloud import storage  
  
app = Flask(__name__)  
  
@app.route('/')  
def index():  
    return render_template('index.html')  
  
@app.route('/upload', methods=['POST'])  
def upload():  
    file = request.files['file']  
    filename = file.filename  
    bucket = storage.Client().bucket('your-bucket-name')  
    blob = bucket.blob(filename)  
    blob.upload_from_string(  
        file.read(),  
        content_type=file.content_type  
    )  
    return redirect(url_for('index'))  
  
if __name__ == '__main__':  
    app.run(debug=True)
```

將上述程式碼儲存為 IDE 工作區根目錄中的 **app.py**。

我們希望應用程式在通訊埠 8080 上執行，並使用主機位址 0.0.0.0，因此請提示 Gemini 修改 **app.run** 陳述式。

輸入下列提示詞：

Modify the app.py to run on port 8080 and host address 0.0.0.0

程式碼現在應如下所示：

```
from flask import Flask, render_template, request, redirect, url_for
from google.cloud import storage

app = Flask(__name__)

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/upload', methods=['POST'])
def upload():
    file = request.files['file']
    filename = file.filename
    bucket = storage.Client().bucket('your-bucket-name')
    blob = bucket.blob(filename)
    blob.upload_from_string(
        file.read(),
        content_type=file.content_type
    )
    return redirect(url_for('index'))

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=8080)
```

接下來，我們也希望系統產生 **index.html**。在 Gemini Chat 視窗中輸入下列提示：

Provide the index.html file

且應該會提供 index.html 的程式碼，如下所示。

```
<!DOCTYPE html>
<html>
<head>
  <title>File Upload</title>
</head>
<body>
  <h1>File Upload</h1>
  <form action="/upload" method="post" enctype="multipart/form-data">
    <input type="file" name="file">
    <input type="submit" value="Upload">
  </form>
</body>
</html>
```

我們可以請 Gemini 告訴我們 **index.html** 檔案的儲存位置。輸入下列提示詞：

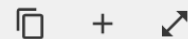
Since this is using the `render_template` framework, where should this file be saved?

回覆內容大致如下：

The `index.html` file should be saved in the `templates` directory within your Flask application directory.

The `render_template` function looks for templates in the `templates` directory by default. So, in this case, the `index.html` file should be saved at the following location:

`/home/romin/webhook-2/templates/index.html`



請在根資料夾中建立名為 `templates` 的資料夾，然後將 `index.html` 檔案儲存到該資料夾。

我們希望在 `app.py` 檔案中加入一些例外狀況處理機制，以防檔案無法儲存至 Google Cloud Storage 值區。保持開啟 `app.py` 檔案，並在 Gemini 對話視窗中輸入下列提示。

modify the `app.py` to have exception handling in the `upload()` function. The success or failure should be returned to the index page.

現在應該會新增一些例外狀況處理方式，如下所示：


```

from flask import Flask, render_template, request, redirect, url_for
from google.cloud import storage

app = Flask(__name__)

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/upload', methods=['POST'])
def upload():
    try:
        file = request.files['file']
        filename = file.filename
        bucket = storage.Client().bucket('your-bucket-name')
        blob = bucket.blob(filename)
        blob.upload_from_string(
            file.read(),
            content_type=file.content_type
        )
        return redirect(url_for('index'))
    except Exception as e:
        return render_template('index.html', error=str(e))

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=8080)

```

由於我們需要在 index.html 中顯示錯誤訊息，因此也需要修改該檔案。請使用下列提示詞要求 Gemini 執行這項操作：

```
update the index.html to display the error message
```

這時應該會產生更新後的 **index.html**，如下所示：

```

<!DOCTYPE html>
<html>
<head>
  <title>File Upload</title>
</head>
<body>
  <h1>File Upload</h1>
  {% if error %}
    <p style="color: red;">{{ error }}</p>
  {% endif %}
  <form action="/upload" method="post" enctype="multipart/form-data">
    <input type="file" name="file">
    <input type="submit" value="Upload">
  </form>
</body>
</html>

```

請務必在上述每個步驟中，儲存 `app.py` 和 `index.html` 檔案中的變更。

`app.py` 沒有正確的 bucket 名稱，因此我們可以將該資訊提供給 Gemini，並要求 Gemini 進行變更。此外，我們必須提供 `storage.Client()` 執行個體的專案 ID。因此，請在 Gemini Chat 視窗中提供下列幾項提示 (將 `<PROJECT_ID>` 替換為您的 Google Cloud 專案 ID)，並納入變更：

提示 1

My bucket name is gemini-for-devs-demo_uploads, please change the code to use that.

提示 2

My project id is gemini-for-devs-demo, please change the storage.Client() to use that.

最終的 `app.py` 檔案如下所示 (下方顯示的是我的專案 ID，但理想上應該是您使用的 ID，也就是您在上述提示中提供的 ID)：

```
from flask import Flask, render_template, request, redirect, url_for
from google.cloud import storage

app = Flask(__name__)

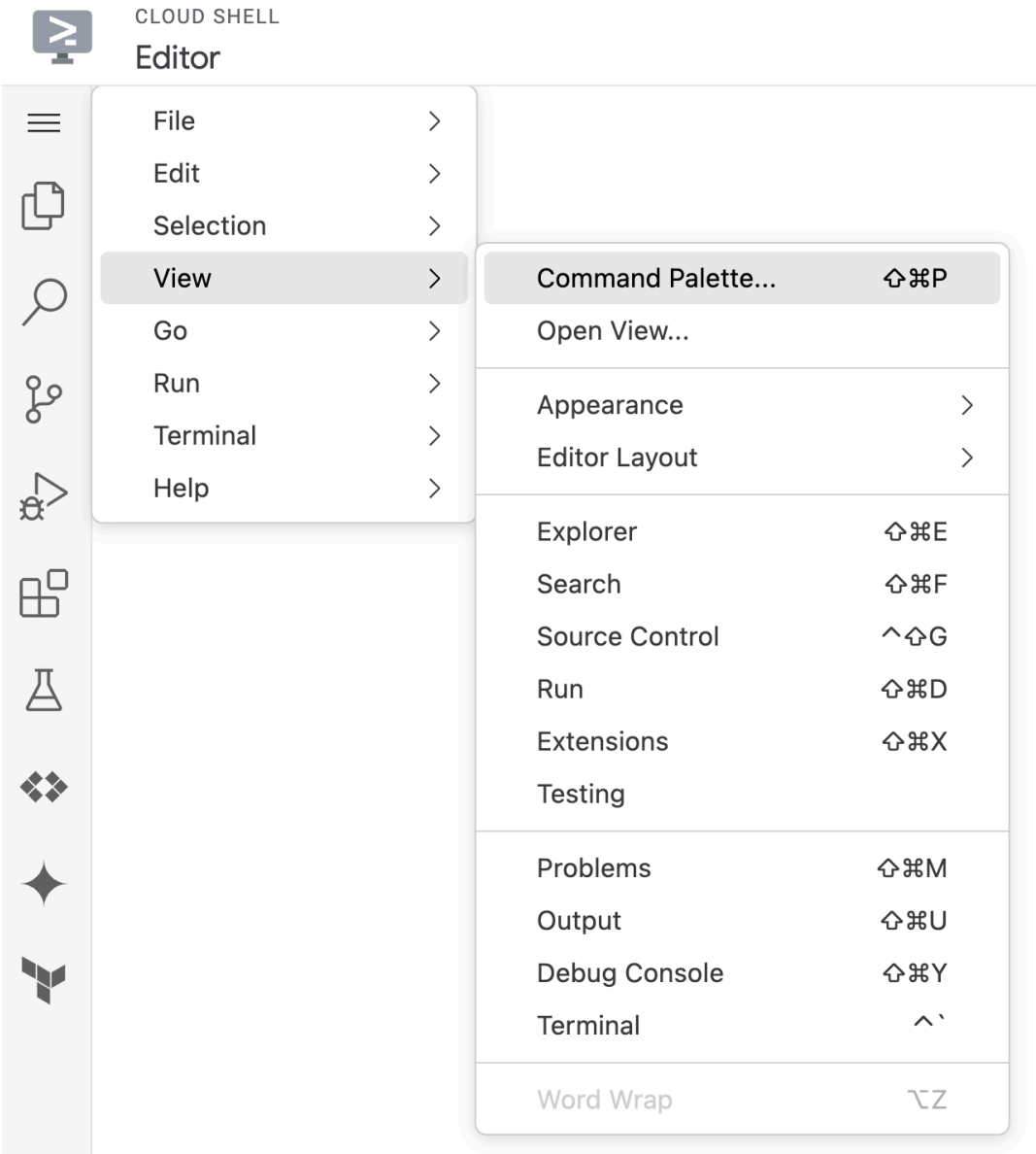
@app.route('/')
def index():
    return render_template('index.html')

@app.route('/upload', methods=['POST'])
def upload():
    try:
        file = request.files['file']
        filename = file.filename
        bucket = storage.Client(project='gcp-experiments-349209').bucket('gcp-experiments-349209_uploads')
        blob = bucket.blob(filename)
        blob.upload_from_string(
            file.read(),
            content_type=file.content_type
        )
        return redirect(url_for('index'))
    except Exception as e:
        return render_template('index.html', error=str(e))

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=8080)
```

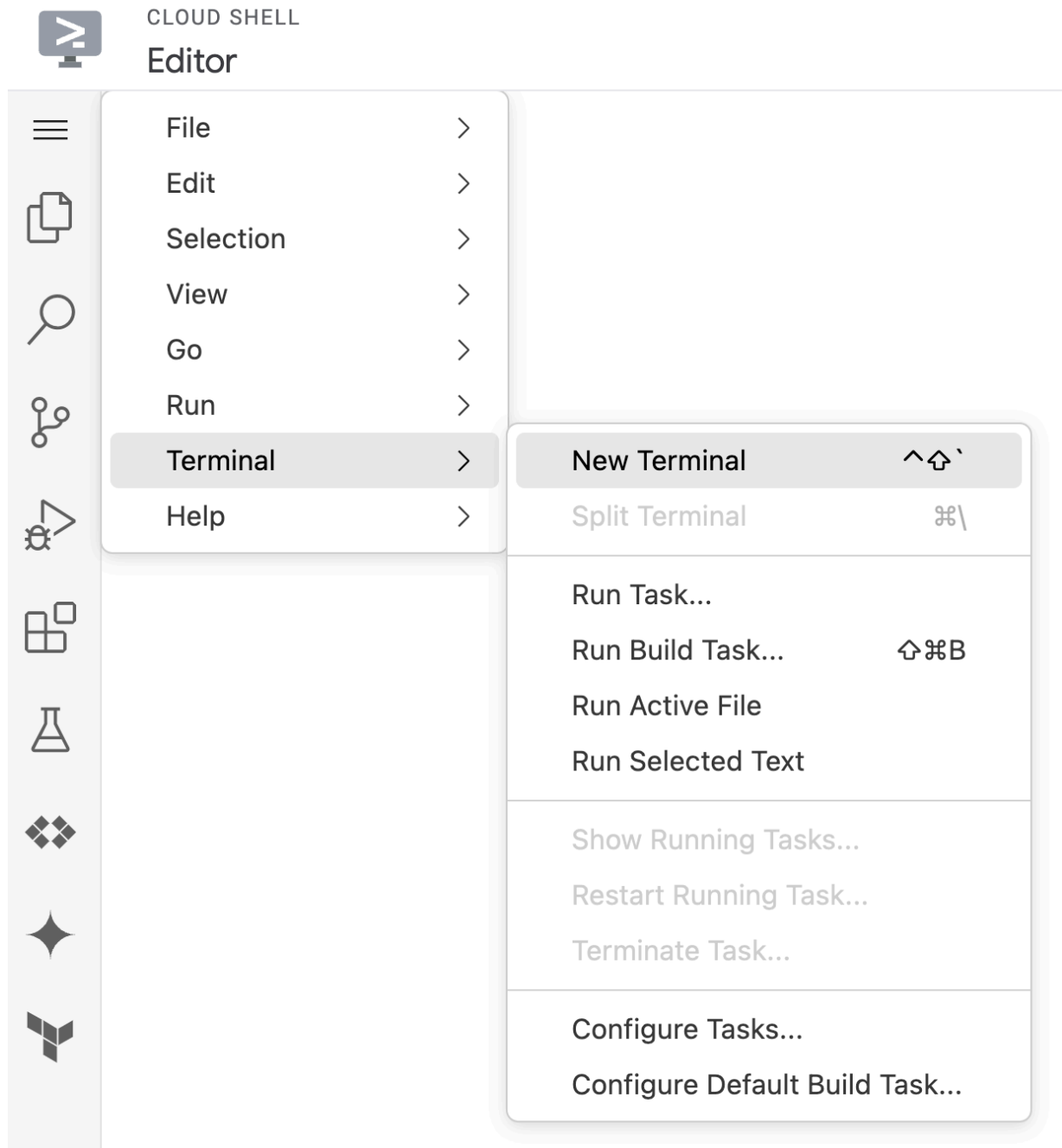
9. 在本機執行網頁應用程式

使用 `requirements.txt` 檔案中定義的依附元件建立 Python 環境。前往 Cloud Shell IDE 中的指令區塊面板，如下所示：



輸入 `Python: Create Environment`，然後按照步驟使用 (venv)、Python 3.x 解譯器和 `requirements.txt` 檔案建立虛擬環境。系統會建立必要環境。

現在啟動終端機，如下所示：



在終端機中輸入下列指令：

```
python app.py
```

Flask 應用程式應會啟動，畫面應如下所示：

```
(.venv) romin@cloudshell:~/webhook-2 (gcp-experiments-349209)$ python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a
production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8080
* Running on http://10.88.0.3:8080
Press CTRL+C to quit
* Restarting with watchdog (inotify)
* Debugger is active!
* Debugger PIN: 989-296-833
```

前往 <http://127.0.0.1:8080> 網址，應該會顯示 `index.html` 頁面

從本機電腦上傳檔案，系統應可成功處理。

您可以前往實驗室稍早看到的 BigQuery 資料集和資料表，查看摘要。或者，您也可以查看 Cloud Storage bucket (`<PROJECT_ID>_output`)。

步驟 10 · 約 5 分鐘

10. (選用) 開放式探索 - 部署至 Cloud Run

- 您可以將應用程式部署至 Cloud Run。
- 使用下列提示詞詢問 Gemini Code Assist (可能需要嘗試上述提示的幾種變體)：

```
I don't want to build a container image but deploy directly from source. What is the gcloud command for that?
```

11. (選用) 開啟「探索 - 新增 CSS 樣式」

- 使用 Gemini Code Assist 和編輯器內建的助理，為應用程式新增 CSS 樣式，完成後再次部署應用程式！
 - 開啟 `index.html` 檔案，並在 Gemini Chat 中輸入下列提示：`Can you apply material design styles to this index.html?`
 - 查看程式碼，確認是否正常運作。
-

步驟 12 · 約 1 分鐘

12. 恭喜！

恭喜！您已成功在範例專案中使用 Gemini Code Assist，瞭解這項工具如何協助您生成、完成及摘要程式碼，並回答 Google Cloud 相關問題。

步驟 13 · 約 0 分鐘

13. 參考文件

- [Gemini Code Assist 首頁](#)
 - [使用 Gemini 進行 AI 輔助和開發 - 說明文件](#)
 - [Gemini 應用實例](#)
-